



+ Code + Text



T4

RAM

Disk

Colab AI



```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import cifar10

# Load CIFAR-10 data
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Resize images to the size expected by the transformer (e.g., 72x72)
size = 72 # You can adjust the size
x_train = tf.image.resize(x_train, (size, size))
x_test = tf.image.resize(x_test, (size, size))

# Normalize pixel values

x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
# Convert class vectors to binary class matrices (one-hot encoding)
num_classes = 10
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 [=====] - 14s 0us/step

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Adjust the number of classes for CIFAR-10
num_classes = 10
input_shape = (32, 32, 3)

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

print(f"x_train shape: {x_train.shape} - y_train shape: {y_train.shape}")
print(f"x_test shape: {x_test.shape} - y_test shape: {y_test.shape}")

# Model configuration variables
learning_rate = 0.001
weight_decay = 0.0001
batch_size = 256 # Same as CIFAR-100
num_epochs = 50 # Increased for better training
image_size = 72 # Resize input images to this size
patch_size = 6 # Size of patches extracted from input images
num_patches = (image_size // patch_size) ** 2
projection_dim = 64
num_heads = 4
transformer_units = [projection_dim * 2, projection_dim]
transformer_layers = 8
mlp_head_units = [2048, 1024]

# Data augmentation pipeline
data_augmentation = keras.Sequential(
    [
        layers.Rescaling(1./255), # Normalize pixel values
        layers.Resizing(image_size, image_size),
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.02),
        layers.RandomZoom(height_factor=0.2, width_factor=0.2),
    ],
    name="data_augmentation",
)

def mlp(x, hidden_units, dropout_rate):
    for units in hidden_units:
        x = layers.Dense(units, activation=keras.activations.gelu)(x)
        x = layers.Dropout(dropout_rate)(x)
    return x
```

```

# Patches and PatchEncoder classes remain unchanged

def create_vit_classifier():
    inputs = keras.Input(shape=input_shape)
    # Augment data.
    augmented = data_augmentation(inputs)
    # Create patches.
    patches = Patches(patch_size)(augmented)
    # Encode patches.
    encoded_patches = PatchEncoder(num_patches, projection_dim)(patches)
    # Create multiple layers of the Transformer block.
    for _ in range(transformer_layers):
        # Layer normalization 1.
        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
        # Create a multi-head attention layer.
        attention_output = layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=projection_dim, dropout=0.1
        )(x1, x1)
        # Skip connection 1.
        x2 = layers.Add()([attention_output, encoded_patches])
        # Layer normalization 2.
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        # MLP.
        x3 = mlp(x3, hidden_units=transformer_units, dropout_rate=0.1)
        # Skip connection 2.
        encoded_patches = layers.Add()([x3, x2])

    # Create a [batch_size, projection_dim] tensor.
    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)
    # Add MLP.
    features = mlp(representation, hidden_units=mlp_head_units, dropout_rate=0.5)
    # Classify outputs.
    logits = layers.Dense(num_classes)(features)
    # Create and return the Keras model
    model = keras.Model(inputs=inputs, outputs=logits)
    return model

def run_experiment(model):
    optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

    model.compile(
        optimizer=optimizer,
        loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=[
            keras.metrics.SparseCategoricalAccuracy(name="accuracy"),
            keras.metrics.SparseTopK_categoricalAccuracy(5, name="top-5-accuracy"),
        ],
    )

    checkpoint_filepath = "/tmp/checkpoint.weights.h5"
    checkpoint_callback = keras.callbacks.ModelCheckpoint(
        checkpoint_filepath,
        monitor="val_accuracy",
        save_best_only=True,
        save_weights_only=True,
    )

    history = model.fit(
        x=x_train,
        y=y_train,
        batch_size=batch_size,
        epochs=num_epochs,
        validation_split=0.1,
        callbacks=[checkpoint_callback],
    )

    model.load_weights(checkpoint_filepath)
    _, accuracy, top_5_accuracy = model.evaluate(x_test, y_test)
    print(f"Test accuracy: {round(accuracy * 100, 2)}%")
    print(f"Test top 5 accuracy: {round(top_5_accuracy * 100, 2)}%")

    return history

# Create and run the model
vit_classifier = create_vit_classifier()
history = run_experiment(vit_classifier)

def plot_history(item):
    plt.plot(history.history[item], label=item)
    plt.plot(history.history["val_" + item], label="val_" + item)

```

```

plt.xlabel("Epochs")
plt.ylabel(item)
plt.title("Train and Validation {} Over Epochs".format(item), fontsize=14)
plt.legend()
plt.grid()
plt.show()

plot_history("loss")
plot_history("top-5-accuracy")

```

```

x_train shape: (50000, 32, 32, 3) - y_train shape: (50000, 1)
x_test shape: (10000, 32, 32, 3) - y_test shape: (10000, 1)
Epoch 1/50
176/176 [=====] - 104s 427ms/step - loss: 2.5467 - accuracy: 0.1534 - top-5-accuracy: 0.6403 - val_loss: 2.0450 - va
Epoch 2/50
176/176 [=====] - 77s 436ms/step - loss: 2.0219 - accuracy: 0.2274 - top-5-accuracy: 0.7659 - val_loss: 1.9329 - val
Epoch 3/50
176/176 [=====] - 77s 436ms/step - loss: 1.8922 - accuracy: 0.2766 - top-5-accuracy: 0.8174 - val_loss: 1.7316 - val
Epoch 4/50
176/176 [=====] - 71s 405ms/step - loss: 1.7861 - accuracy: 0.3254 - top-5-accuracy: 0.8474 - val_loss: 1.7503 - val
Epoch 5/50
176/176 [=====] - 77s 437ms/step - loss: 1.7127 - accuracy: 0.3619 - top-5-accuracy: 0.8640 - val_loss: 1.5701 - val
Epoch 6/50
176/176 [=====] - 77s 436ms/step - loss: 1.6164 - accuracy: 0.4000 - top-5-accuracy: 0.8874 - val_loss: 1.4344 - val
Epoch 7/50
176/176 [=====] - 74s 421ms/step - loss: 1.5350 - accuracy: 0.4354 - top-5-accuracy: 0.9014 - val_loss: 1.3760 - val
Epoch 8/50
176/176 [=====] - 79s 451ms/step - loss: 1.4790 - accuracy: 0.4642 - top-5-accuracy: 0.9103 - val_loss: 1.3675 - val
Epoch 9/50
176/176 [=====] - 77s 440ms/step - loss: 1.4246 - accuracy: 0.4889 - top-5-accuracy: 0.9163 - val_loss: 1.2905 - val
Epoch 10/50
176/176 [=====] - 77s 439ms/step - loss: 1.3674 - accuracy: 0.5131 - top-5-accuracy: 0.9252 - val_loss: 1.2813 - val
Epoch 11/50
176/176 [=====] - 80s 452ms/step - loss: 1.3129 - accuracy: 0.5318 - top-5-accuracy: 0.9327 - val_loss: 1.2112 - val
Epoch 12/50
176/176 [=====] - 77s 438ms/step - loss: 1.2740 - accuracy: 0.5460 - top-5-accuracy: 0.9379 - val_loss: 1.1990 - val
Epoch 13/50
176/176 [=====] - 77s 440ms/step - loss: 1.2291 - accuracy: 0.5635 - top-5-accuracy: 0.9442 - val_loss: 1.1281 - val
Epoch 14/50
176/176 [=====] - 78s 441ms/step - loss: 1.1898 - accuracy: 0.5774 - top-5-accuracy: 0.9483 - val_loss: 1.0553 - val
Epoch 15/50
176/176 [=====] - 78s 441ms/step - loss: 1.1452 - accuracy: 0.5934 - top-5-accuracy: 0.9525 - val_loss: 1.0220 - val
Epoch 16/50
176/176 [=====] - 78s 441ms/step - loss: 1.1081 - accuracy: 0.6099 - top-5-accuracy: 0.9551 - val_loss: 0.9742 - val
Epoch 17/50
176/176 [=====] - 78s 441ms/step - loss: 1.0650 - accuracy: 0.6248 - top-5-accuracy: 0.9608 - val_loss: 0.9501 - val
Epoch 18/50
176/176 [=====] - 80s 452ms/step - loss: 1.3129 - accuracy: 0.5318 - top-5-accuracy: 0.9327 - val_loss: 1.2112 - val
Epoch 12/50
176/176 [=====] - 77s 438ms/step - loss: 1.2740 - accuracy: 0.5460 - top-5-accuracy: 0.9379 - val_loss: 1.1990 - val
Epoch 13/50
176/176 [=====] - 77s 440ms/step - loss: 1.2291 - accuracy: 0.5635 - top-5-accuracy: 0.9442 - val_loss: 1.1281 - val
Epoch 14/50
176/176 [=====] - 78s 441ms/step - loss: 1.1898 - accuracy: 0.5774 - top-5-accuracy: 0.9483 - val_loss: 1.0553 - val
Epoch 15/50
176/176 [=====] - 78s 441ms/step - loss: 1.1452 - accuracy: 0.5934 - top-5-accuracy: 0.9525 - val_loss: 1.0220 - val
Epoch 16/50
176/176 [=====] - 78s 441ms/step - loss: 1.1081 - accuracy: 0.6099 - top-5-accuracy: 0.9551 - val_loss: 0.9742 - val
Epoch 17/50
176/176 [=====] - 78s 441ms/step - loss: 1.0650 - accuracy: 0.6248 - top-5-accuracy: 0.9608 - val_loss: 0.9501 - val
Epoch 18/50
176/176 [=====] - 76s 430ms/step - loss: 1.0318 - accuracy: 0.6346 - top-5-accuracy: 0.9640 - val_loss: 0.9174 - val
Epoch 19/50
176/176 [=====] - 77s 436ms/step - loss: 1.0066 - accuracy: 0.6432 - top-5-accuracy: 0.9657 - val_loss: 0.9116 - val
Epoch 20/50
176/176 [=====] - 77s 439ms/step - loss: 0.9857 - accuracy: 0.6524 - top-5-accuracy: 0.9681 - val_loss: 0.8468 - val
Epoch 21/50
176/176 [=====] - 71s 406ms/step - loss: 0.9407 - accuracy: 0.6669 - top-5-accuracy: 0.9714 - val_loss: 0.8995 - val
Epoch 22/50
176/176 [=====] - 80s 454ms/step - loss: 0.9203 - accuracy: 0.6737 - top-5-accuracy: 0.9735 - val_loss: 0.8254 - val
Epoch 23/50
176/176 [=====] - 78s 441ms/step - loss: 0.8972 - accuracy: 0.6832 - top-5-accuracy: 0.9744 - val_loss: 0.7864 - val
Epoch 24/50
176/176 [=====] - 72s 407ms/step - loss: 0.8742 - accuracy: 0.6914 - top-5-accuracy: 0.9758 - val_loss: 0.8137 - val
Epoch 25/50
176/176 [=====] - 80s 455ms/step - loss: 0.8587 - accuracy: 0.6962 - top-5-accuracy: 0.9758 - val_loss: 0.7901 - val
Epoch 26/50
176/176 [=====] - 78s 441ms/step - loss: 0.8340 - accuracy: 0.7077 - top-5-accuracy: 0.9780 - val_loss: 0.7661 - val
Epoch 27/50
176/176 [=====] - 77s 440ms/step - loss: 0.8092 - accuracy: 0.7137 - top-5-accuracy: 0.9800 - val_loss: 0.7438 - val
Epoch 28/50
176/176 [=====] - 72s 407ms/step - loss: 0.8000 - accuracy: 0.7199 - top-5-accuracy: 0.9808 - val_loss: 0.7488 - val
Epoch 29/50
176/176 [=====] - 79s 449ms/step - loss: 0.7801 - accuracy: 0.7238 - top-5-accuracy: 0.9821 - val_loss: 0.7541 - val
Epoch 30/50
176/176 [=====] - 77s 437ms/step - loss: 0.7654 - accuracy: 0.7291 - top-5-accuracy: 0.9827 - val_loss: 0.7229 - val
Epoch 31/50
176/176 [=====] - 77s 440ms/step - loss: 0.7482 - accuracy: 0.7338 - top-5-accuracy: 0.9838 - val_loss: 0.7076 - val

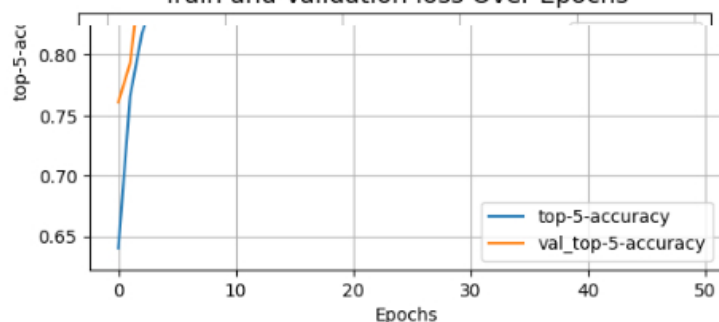
```

```

Epoch 32/50
176/176 [=====] - 72s 407ms/step - loss: 0.7191 - accuracy: 0.7477 - top-5-accuracy: 0.9856 - val_loss: 0.7184 - val
Epoch 33/50
176/176 [=====] - 80s 453ms/step - loss: 0.7097 - accuracy: 0.7489 - top-5-accuracy: 0.9858 - val_loss: 0.7037 - val
Epoch 34/50
176/176 [=====] - 77s 441ms/step - loss: 0.6823 - accuracy: 0.7568 - top-5-accuracy: 0.9872 - val_loss: 0.7091 - val
Epoch 35/50
176/176 [=====] - 72s 407ms/step - loss: 0.6775 - accuracy: 0.7605 - top-5-accuracy: 0.9869 - val_loss: 0.7279 - val
Epoch 36/50
176/176 [=====] - 77s 439ms/step - loss: 0.6688 - accuracy: 0.7624 - top-5-accuracy: 0.9876 - val_loss: 0.6908 - val
Epoch 37/50
176/176 [=====] - 71s 406ms/step - loss: 0.6469 - accuracy: 0.7700 - top-5-accuracy: 0.9884 - val_loss: 0.6825 - val
Epoch 38/50
176/176 [=====] - 77s 440ms/step - loss: 0.8092 - accuracy: 0.7137 - top-5-accuracy: 0.9800 - val_loss: 0.7438 - val
Epoch 28/50
176/176 [=====] - 72s 407ms/step - loss: 0.8000 - accuracy: 0.7199 - top-5-accuracy: 0.9808 - val_loss: 0.7488 - val
Epoch 29/50
176/176 [=====] - 79s 449ms/step - loss: 0.7801 - accuracy: 0.7238 - top-5-accuracy: 0.9821 - val_loss: 0.7541 - val
Epoch 30/50
176/176 [=====] - 77s 437ms/step - loss: 0.7654 - accuracy: 0.7291 - top-5-accuracy: 0.9827 - val_loss: 0.7229 - val
Epoch 31/50
176/176 [=====] - 77s 440ms/step - loss: 0.7482 - accuracy: 0.7338 - top-5-accuracy: 0.9838 - val_loss: 0.7076 - val
Epoch 32/50
176/176 [=====] - 72s 407ms/step - loss: 0.7191 - accuracy: 0.7477 - top-5-accuracy: 0.9856 - val_loss: 0.7184 - val
Epoch 33/50
176/176 [=====] - 80s 453ms/step - loss: 0.7097 - accuracy: 0.7489 - top-5-accuracy: 0.9858 - val_loss: 0.7037 - val
Epoch 34/50
176/176 [=====] - 77s 441ms/step - loss: 0.6823 - accuracy: 0.7568 - top-5-accuracy: 0.9872 - val_loss: 0.7091 - val
Epoch 35/50
176/176 [=====] - 72s 407ms/step - loss: 0.6775 - accuracy: 0.7605 - top-5-accuracy: 0.9869 - val_loss: 0.7279 - val
Epoch 36/50
176/176 [=====] - 77s 439ms/step - loss: 0.6688 - accuracy: 0.7624 - top-5-accuracy: 0.9876 - val_loss: 0.6908 - val
Epoch 37/50
176/176 [=====] - 71s 406ms/step - loss: 0.6469 - accuracy: 0.7700 - top-5-accuracy: 0.9884 - val_loss: 0.6825 - val
Epoch 38/50
176/176 [=====] - 74s 419ms/step - loss: 0.6254 - accuracy: 0.7784 - top-5-accuracy: 0.9896 - val_loss: 0.6934 - val
Epoch 39/50
176/176 [=====] - 80s 453ms/step - loss: 0.6126 - accuracy: 0.7835 - top-5-accuracy: 0.9895 - val_loss: 0.6542 - val
Epoch 40/50
176/176 [=====] - 72s 408ms/step - loss: 0.5954 - accuracy: 0.7886 - top-5-accuracy: 0.9911 - val_loss: 0.6635 - val
Epoch 41/50
176/176 [=====] - 77s 439ms/step - loss: 0.5961 - accuracy: 0.7890 - top-5-accuracy: 0.9906 - val_loss: 0.6529 - val
Epoch 42/50
176/176 [=====] - 71s 406ms/step - loss: 0.5768 - accuracy: 0.7965 - top-5-accuracy: 0.9914 - val_loss: 0.6400 - val
Epoch 43/50
176/176 [=====] - 79s 451ms/step - loss: 0.5598 - accuracy: 0.8018 - top-5-accuracy: 0.9920 - val_loss: 0.6425 - val
Epoch 44/50
176/176 [=====] - 72s 408ms/step - loss: 0.5400 - accuracy: 0.8088 - top-5-accuracy: 0.9926 - val_loss: 0.6377 - val
Epoch 45/50
176/176 [=====] - 80s 454ms/step - loss: 0.5272 - accuracy: 0.8125 - top-5-accuracy: 0.9931 - val_loss: 0.6356 - val
Epoch 46/50
176/176 [=====] - 72s 407ms/step - loss: 0.5207 - accuracy: 0.8165 - top-5-accuracy: 0.9934 - val_loss: 0.6260 - val
Epoch 47/50
176/176 [=====] - 74s 419ms/step - loss: 0.5077 - accuracy: 0.8195 - top-5-accuracy: 0.9936 - val_loss: 0.6228 - val
Epoch 48/50
176/176 [=====] - 71s 406ms/step - loss: 0.4977 - accuracy: 0.8237 - top-5-accuracy: 0.9942 - val_loss: 0.6420 - val
Epoch 49/50
176/176 [=====] - 74s 418ms/step - loss: 0.4838 - accuracy: 0.8302 - top-5-accuracy: 0.9941 - val_loss: 0.6465 - val
Epoch 50/50
176/176 [=====] - 71s 405ms/step - loss: 0.4801 - accuracy: 0.8302 - top-5-accuracy: 0.9939 - val_loss: 0.6380 - val
313/313 [=====] - 7s 23ms/step - loss: 0.6429 - accuracy: 0.7767 - top-5-accuracy: 0.9873
Test accuracy: 77.67%
Test top 5 accuracy: 98.73%

```

Train and Validation loss Over Epochs



```

# Comparing the DL-based image classification algorithm in Q2 with Vision Transformer in Q4,
# the algorithm from Q2 yielded a 74.10% test accuracy from 20 epochs while the ViT achieved
# a test accuracy of 77.67%. The ViT attains a higher score in terms of exact label prediction
# accuracy. This shows that the ViT algorithm is doing better in complex pattern if though it is
# using more data and much more ram compared to CNNs. The ViT also yielded a 98.77% top-5 accuracy
# which is a typical high accuracy when prediction fewer classes, but still indicates correct class label
# predictions.

```

[] Start coding or [generate](#) with AI.

[Colab paid products](#) - [Cancel contracts here](#)

✓ 0s completed at 10:03PM

